

AI & LLM Learning Roadmap

From zero to understanding large language models

6 Layers • Complete Guide

How to use this roadmap

This roadmap is structured as six layers, each one building on the previous. You do not need to master a layer completely before moving to the next — aim for solid understanding and move forward. You will revisit earlier layers naturally as later concepts demand it.

Suggested approach

Start at Layer 1. Do not skip it even if you have some math background.
For each topic: read the concept, write code to test it, then explain it out loud.
When you hit a wall in a later layer, return to the relevant earlier section.
The end goal is to read a paper like TurboQuant and follow every step.

At a glance

Layer	Name	Core skill you gain	Timeline
1	Math foundations	Read and write the equations behind ML	4–8 weeks
2	Classical ML	Understand how models learn from data	3–5 weeks

Layer	Name	Core skill you gain	Timeline
3	Neural networks	Build and train networks from scratch	4–6 weeks
4	The transformer	Understand the architecture behind all LLMs	4–6 weeks
5	LLMs specifically	Know how GPT, Llama, and Claude work	3–4 weeks
6	Production & efficiency	Quantization, KV cache, deployment	3–5 weeks

Layer 1 Math Foundations

The language everything else is written in

Why this layer matters

Every operation in machine learning is, at its core, a mathematical operation on vectors and matrices. Skipping this layer means you will hit a wall the moment you try to read a research paper or debug a real implementation. The good news: you do not need a maths degree. You need working fluency with four topics.

1.1 Linear algebra

Linear algebra is the foundation. Neural networks are, literally, chains of matrix multiplications. When a transformer processes your text, it is doing hundreds of matrix multiplications per token. When TurboQuant rotates a vector, that is a matrix multiply. You cannot avoid it.

Topics to cover:

- Vectors and what they represent — a list of numbers encoding position, meaning, or state
- Vector operations — addition, scaling, dot product, L2 norm
- Matrices — what they are and how matrix multiplication works
- The dot product deeply — why it measures similarity between vectors
- Matrix transpose, inverse, and identity matrix
- Eigenvalues and eigenvectors — what they reveal about a transformation
- Orthogonal matrices — why rotation preserves distances (key for TurboQuant)
- QR decomposition — how to produce a random rotation matrix

Why orthogonal matrices matter for TurboQuant

The very first step of TurboQuant is to multiply the input vector by a random orthogonal matrix. An orthogonal matrix Q satisfies $Q^T * Q = I$, meaning it rotates without stretching. This is how TurboQuant turns any input into something with a known, uniform distribution. You cannot understand this without knowing what matrix multiplication does geometrically.

1.2 Calculus

Calculus tells you how things change. In ML, the core operation is: given a function that measures error, find the input values that minimize it. That process is gradient descent, and it is built entirely from derivatives.

Topics to cover:

- Derivatives — what they measure, how to compute them

- The chain rule — essential for backpropagation through layers
- Partial derivatives — derivatives when you have many inputs
- Gradients — the multi-dimensional version of a derivative
- The gradient as a direction of steepest ascent
- Minima and maxima — how to find where a function is smallest

1.3 Probability and statistics

ML models are fundamentally probabilistic. They output distributions, are trained on random samples, and are evaluated statistically. TurboQuant's guarantees are all stated as expected values and variances — you need probability to understand what those mean.

Topics to cover:

- Random variables, probability distributions, PDFs and CDFs
- The Gaussian (normal) distribution and why it appears everywhere
- The Beta distribution — directly relevant to TurboQuant's coordinate analysis
- Expected value (mean) and variance
- The law of large numbers and central limit theorem
- Conditional probability and Bayes' theorem
- What it means for an estimator to be unbiased

1.4 Information theory

Information theory is the science of compression and communication. It gives us the theoretical limits on how much we can compress data. TurboQuant is directly motivated by Shannon's source coding theory — the paper cites Shannon's original work repeatedly and proves bounds using Shannon's lower bound formula.

Topics to cover:

- Entropy — how to measure the information content of a distribution
- Shannon's source coding theorem — the fundamental limit on compression
- Mutual information — how much two random variables share
- The distortion-rate function — the tradeoff between compression and error
- Why these limits cannot be beaten by any algorithm

Resources for Layer 1

Recommended resources

Linear algebra: 3Blue1Brown 'Essence of Linear Algebra' series on YouTube (free, visual, excellent)

Linear algebra (deeper): Gilbert Strang's MIT 18.06 lectures (free on MIT OpenCourseWare)

Calculus: Khan Academy calculus series (free, thorough, well-paced)

Probability: 'Introduction to Probability' by Blitzstein & Hwang (free PDF online)
Information theory: Chapter 2 of 'Elements of Information Theory' by Cover & Thomas
Practical: implement everything in NumPy — coding forces real understanding

Layer 2 Classical ML

How models learn from data

Why this layer matters

Before deep learning, ML researchers developed clean, principled approaches to learning from data. These ideas — loss functions, optimization, overfitting, generalization — are still the core concepts in modern AI. Deep learning is an extension of these ideas, not a replacement.

2.1 Linear and logistic regression

These are the simplest models that actually learn. They teach you the fundamental pattern: define a model, define a loss, minimize the loss. Every neural network follows this same pattern.

- Linear regression — fitting a line to data by minimizing squared error
- Loss functions — what we are minimizing and why the choice matters
- The normal equation — closed-form solution via linear algebra
- Logistic regression — extending to classification with probabilities
- Cross-entropy loss — the standard loss for classification

2.2 Gradient descent

Gradient descent is the engine of all modern ML. The idea is simple: compute the gradient of your loss with respect to your parameters, then take a small step in the opposite direction. Repeat until convergence. Understanding this deeply is non-negotiable.

- Batch gradient descent — use all data to compute each update
- Stochastic and mini-batch gradient descent — practical variants
- Learning rate — the most important hyperparameter
- Momentum, Adam, and adaptive optimizers — why plain gradient descent is not enough
- The loss landscape — what you are actually navigating

2.3 Overfitting and generalization

A model that memorizes training data is useless. Understanding why models generalize — and when they fail to — is one of the most important skills in ML.

- Train vs validation vs test sets — why you need three splits
- Overfitting and underfitting — diagnosing from learning curves
- Regularization — L1, L2, dropout
- Bias-variance tradeoff — the fundamental tension in model design

2.4 Dimensionality and embeddings

High-dimensional spaces behave counterintuitively. This topic prepares you for understanding why random rotations in TurboQuant work, and why embeddings in LLMs are so powerful.

- The curse of dimensionality — why distance breaks down in high dimensions
- PCA — reducing dimensions while preserving variance
- What embeddings are — dense vector representations of discrete objects
- Why similar things cluster together in embedding space

Layer 3 Neural Networks

Learning hierarchical representations

Why this layer matters

Neural networks are the building block of all modern AI. A transformer is a specific kind of neural network. To understand GPT or Llama, you must first understand how a basic network is structured, how it is trained, and what it is actually doing.

3.1 Network structure

- Neurons, weights, and biases — the basic computational unit
- Activation functions — ReLU, sigmoid, tanh, and why they are needed
- Layers and depth — what deep means and why it helps
- The forward pass — how input becomes output
- Universal approximation — why neural networks can represent any function

3.2 Backpropagation

Backpropagation is how neural networks learn. It is the chain rule applied repeatedly through a network to compute gradients. Understanding this deeply means you will never be confused by training dynamics.

- Computational graphs — representing operations as a graph
- Forward and backward passes — what happens during training
- Vanishing and exploding gradients — why deep networks were hard to train
- Batch normalization — stabilizing training in deep networks

3.3 Common architectures

- Fully connected networks — the baseline
- Convolutional neural networks (CNNs) — for spatial data like images
- Recurrent neural networks (RNNs) and LSTMs — for sequences
- Why RNNs struggle with long sequences — the motivation for transformers

Layer 4 The Transformer

The architecture behind every modern LLM

Why this layer matters

The transformer, introduced in 'Attention is All You Need' (2017), replaced RNNs for almost every sequence task. Every major LLM today — GPT-4, Claude, Llama, Gemini — is a transformer. You cannot understand LLMs without understanding transformers.

4.1 The attention mechanism

Attention is the core innovation of the transformer. It allows any token to directly attend to any other token in the sequence, regardless of distance. This solved the long-range dependency problem that broke RNNs.

- Queries, keys, and values — the three projections
- Scaled dot-product attention — the core formula
- Attention weights — what they mean and how to read them
- Why attention is $O(n^2)$ in sequence length — and why that matters

4.2 Multi-head attention

- Why one attention head is not enough
- Running attention in parallel across multiple heads
- Concatenating and projecting head outputs
- What different heads learn to attend to

4.3 The full transformer block

- Layer normalization — where it goes and why
- The feedforward sublayer — two linear layers with a nonlinearity
- Residual connections — why they are critical for deep transformers
- Positional encoding — how transformers know token order

4.4 Encoder vs decoder

- Encoder-only models like BERT — bidirectional, good for understanding
- Decoder-only models like GPT — autoregressive, good for generation
- Encoder-decoder models like T5 — translation and summarization
- Why most modern LLMs are decoder-only

Layer 5 LLMs Specifically

How language models are built and trained

Why this layer matters

Understanding the transformer architecture is necessary but not sufficient. LLMs involve a specific set of training choices, data pipelines, and inference procedures that are distinct from general deep learning. This layer covers what makes an LLM an LLM.

5.1 Tokenization

Text must be converted to numbers before a model can process it. The way this is done has a significant effect on model performance.

- Byte-pair encoding (BPE) — the most common tokenization algorithm
- Vocabulary size and its tradeoffs
- Subword tokenization — why whole words are not used
- How tokens map to vector embeddings

5.2 Pre-training

- Next-token prediction — the self-supervised training objective
- Training data — what LLMs are trained on and why it matters
- Scaling laws — why bigger models trained on more data are better
- Compute requirements — what it actually costs to train an LLM

5.3 Fine-tuning and alignment

- Supervised fine-tuning (SFT) — adapting to specific tasks
- RLHF — reinforcement learning from human feedback
- Instruction following — how models learn to follow prompts
- Constitutional AI and direct preference optimization

5.4 Inference and generation

Understanding inference is what connects to Layer 6. This is where the KV cache lives.

- Autoregressive generation — generating one token at a time
- Sampling strategies — temperature, top-k, top-p
- The KV cache — why storing past key/value vectors is necessary
- Why the KV cache grows with context length — the memory bottleneck
- Prefill vs decode phases of inference

Layer 6 Production & Efficiency

Making LLMs fast, small, and deployable

Why this layer matters

LLMs are expensive. A 70B parameter model requires ~140GB of GPU memory in float16. Real deployments require aggressive optimization. This layer is where TurboQuant lives — it is the cutting edge of making inference cheaper without hurting quality.

6.1 Quantization

Quantization converts floating-point weights or activations to lower-precision integers. This is the primary technique for reducing model memory and increasing throughput.

- Float32, float16, bfloat16, int8, int4 — the precision landscape
- Post-training quantization (PTQ) vs quantization-aware training (QAT)
- Weight quantization vs activation quantization — different challenges
- GPTQ, AWQ, and other modern approaches
- TurboQuant — near-optimal vector quantization with theoretical guarantees

6.2 KV cache compression

The KV cache is often the dominant memory cost during long-context inference. Compressing it is one of the most active research areas in LLM efficiency.

- Why the KV cache grows linearly with context length
- Quantization of KV vectors — the TurboQuant use case
- Token eviction — dropping less important past tokens
- Multi-query and grouped-query attention — architectural approaches

6.3 Vector databases and search

Embeddings are used to build searchable indexes over large corpora. Product quantization is the standard approach for compressing those indexes.

- How vector databases store and search embeddings
- Approximate nearest neighbor search — HNSW, IVF
- Product quantization — the traditional approach TurboQuant improves on
- Recall vs latency vs memory tradeoffs

6.4 Hardware and deployment

- GPU memory hierarchy — HBM vs SRAM and why it matters
- Memory bandwidth as the bottleneck during inference

- Tensor parallelism and pipeline parallelism
- Flash Attention — reordering operations to reduce memory movement
- Speculative decoding — using a small model to draft tokens

After Layer 6: reading research papers

With all six layers complete, you will be able to open a paper like TurboQuant and follow the full argument: the problem definition, the algorithm, the proofs, and the experiments. That is the goal of this roadmap.

Papers to read in order

1. 'Attention is All You Need' (Vaswani et al., 2017) — the original transformer
2. 'Language Models are Few-Shot Learners' (Brown et al., 2020) — GPT-3
3. 'Scaling Laws for Neural Language Models' (Kaplan et al., 2020)
4. 'Product Quantization for Nearest Neighbor Search' (Jegou et al., 2011)
5. 'GPTQ: Accurate Post-Training Quantization' (Frantar et al., 2022)
6. 'TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate' (2025)

Estimated timeline

Phase	Topics
Weeks 1-8 Layer 1	Linear algebra, calculus, probability, information theory
Weeks 9-13 Layer 2	Linear regression, gradient descent, overfitting, embeddings
Weeks 14-19 Layer 3	Neural networks, backpropagation, CNNs, RNNs
Weeks 20-25 Layer 4	Attention, multi-head attention, full transformer, encoder vs decoder
Weeks 26-29 Layer 5	Tokenization, pre-training, fine-tuning, inference, KV cache
Weeks 30-34 Layer 6	Quantization, KV cache compression, vector databases, hardware

These are estimates for someone studying part-time (~1-2 hours per day). With more time, each layer can be completed faster. The timeline is less important than the depth — move to the next layer when you can explain the current one, not just when the weeks have passed.